3.页面布局与设备适配

这节课我们介绍css3中与页面布局相关的属性(分栏布局，网格布局，弹性布局)以及设备尺寸适配相关的属性(媒体查询，环境变量函数，vw等)，为了方便讲解，下面内容提及的子项，一般指直接子项

3.1 分栏布局

有别于传统布局，分栏布局会将子元素在内的所有内容拆分为列，文档流呈倒N方向流动，以下属性与分栏布局相关：

- columns
- column-width
- column-count
- column-rule
- column-rule-color
- column-rule-style
- column-rule-width
- column-span
- column-fill
- column-gap
- break-inside

其中使用频率最高的是columns和column-gap，我们重点关注

3.1.1 columns与column-gap

columns是column-width和column-count的缩写，`column-width`表示期望的每列宽度，`column-count`表示期望的列数，其标准语法如下：

CSS

```
columns = <column-width> || <column-count>
column-width = <length> | auto
column-count = <integer> | auto
```

可以看到 column-width 只支持数字加单位的写法，不支持百分比写法，column-count 只支持整数，默认值都为auto，缩写时二者顺序可以互换

上面说到，二者都是期望的值，因为实际的分列效果并不是直接由它们决定的，而是二者“相互制约”，浏览器会参考二者的期望值，来看下例子

https://codepen.io/dqqbl/pen/vYpjPMg

可以看到，前两个例子最终的**列数**取的是两个期望值计算后的最小列数，而第三个例子出现三列而不是四列的原因是 `column-gap` 属性的影响，它表示列间隙，默认值是 `1em`；

我们也可以直接使用gap属性设置分栏间隙大小，原因是column-gap其实是gap属性的子属性

> 在网格布局规范制定之后的一段时间，CSS世界中的行与列之间的间隙使用了gap属性进行了统一，也就是分栏布局、弹性布局和网格布局的间隙都全部统一使用gap属性表示，而gap属性实际上是 column-gap属性和row-gap属性的缩写

3.1.2 其它属性

- column-rule

  该属性用来定义分栏的分割线样式，它是column-rule属性是column-rule-width、column-rule-style和column-rule-color这3个属性的缩写，语法与border一致

https://codepen.io/dqqbl/pen/MWrGRrW

- column-span

  column-span属性有类似于表格布局中的HTML属性colspan， 表示某一个内容是否跨多栏显示，但是它只有两个值

  - none表示不横跨多栏，默认值
  - all表示横跨所有垂直列

https://codepen.io/dqqbl/pen/gOozyzj

该属性可以用来在分栏布局中插入广告🐶

- column-fill

  该属性表示每列的填充方式，取值如下：

  - auto 填满前一列再填充下一列
  - balance 尽量平衡每列间的内容，默认值
  - balance-all 没有浏览器支持，可忽略
- break-inside

  该属性可以定义页面打印、分栏布局或Regions布局(已废弃)发生中断时元素的表现形式。如果没有发生中断，则忽略该属性，由于兼容性等原因，我们目前只需要关注下面两个属性值就好：

  - auto 表示元素可以中断，默认值
  - avoid表示元素不能中断

分栏布局的使用场景有单行内容的两端对齐，和电子书的水平翻页效果

3.2 弹性布局

给任意元素设置display:flex / inline-flex声明即可创建弹性布局，同时会创建FFC(flex formatting context)弹性格式化上下文，在弹性布局容器下有以下特性：

- 弹性布局的子项会被块级化，display值会改变如inline/inline-block会变为block，inline-flex会变为flex，块级化对匿名盒子也生效
- 虽然子项的display值被修改为块级，但它们已经不是块级盒子而是弹性级盒子，因为它们处在FFC中而不是BFC中，子项的默认尺寸表现为收缩而不是独占一行
- flex子项float浮动失效，flex中有左右对齐专用属性
- flex子项支持z-index属性，当z-index不为auto时，会创建层叠上下文
- flex子项的margin值不会合并，原因我们之前也学习过，flex的子项会创建块级格式化上下文(BFC)，BFC之间的margin是不会重叠的
- flex子项的尺寸在底层是有具体计算值的，因此margin: auto 生效
- 绝对定位会使得子项脱离flex布局

https://codepen.io/dqqbl/pen/zYpmQvV

3.2.1 flex-flow

flex-flow

flex-flow标准语法如下：

```
CSS
flex-flow: <'flex-direction'> || <'flex-wrap'>
```

它是flex-direction和flex-wrap的缩写，这两个值无序且非必需，开发时建议直接使用flex-flow，下面是这两个值的解析

flex-direction

flex布局方向由主轴和交叉轴组成，flex-direction用于指定主轴方向，取值：

- row 默认值，主轴为水平方向，流向为当前文档水平流方向，默认为从左到右
- row-reverse 主轴为水平方向，流向与文档水平流相反
- column 主轴为垂直方向，流向为当前文档垂直流方向
- column-reverse 主轴为垂直方向，流向与文档垂直流相反

例子

flex-wrap

用于控制子项换行，取值：

- nowrap 不换行，默认值

- wrap 按当前文档垂直流换行
- wrap-reverse 换行方向与当前文档垂直流相反

例子

下面为了方便讲解，我们默认主轴为水平方向，交叉轴为垂直方向

3.2.2 justify-content

justify-content在flex布局中用于控制主轴内容区的对齐方式，需要注意的是类似于`justify-content`,`align-items`等对齐属性并不是弹性布局专属的，在其它布局中也可以使用，但仅部分值在对应的布局中生效，在flex布局中，它有以下值：

- normal justify-content的初始值，在flex布局下表现为flex-start
- flex-start 可以看作是flex布局下的默认值，它是一个逻辑属性，表示容器的整体内容位于文档流头部(默认为左对齐)
- flex-end 逻辑属性，表示容器的整体内容位于水平文档流尾部(默认为右对齐)
- center 整体内容布局居中
- space-between 表示多余的空白空间只在两子项间的区域均分，space-前缀的属性都用于多余的空白空间的分配
- space-around 每个子项两侧都环绕互不干扰的等宽的空白间距，最终在视觉表现上空白空间比例类似于 1:2:2:2:1
- space-evenly 表示每个flex子项两侧空白间距完全相等，最终在视觉表现上空白空间比例类似于 1:1:1:1:1

例子

3.2.3 align-items 与 align-self

align-items/align-self用于控制交叉轴(默认为垂直方向)子项的对齐方式，align-items设置在容器上，从名字上可以看出，它会对每个子项生效；而align-self设置在子项上，只对设置的子项生效；align-self的默认值是auto，表示跟随容器设置，其余两者支持的值一致：

- stretch 可以看作是弹性布局中align-items属性的默认值，表示子项目在垂直方向上拉伸，子项高度填满所在行，如果子项设置了高度，则以高度值为准
- flex-start/flex-end 逻辑属性，表示子项在所在行内顶部/底部对齐，注意此时不是默认值stretch所以子项不会拉伸
- center 子项在所在行内居中对齐
- baseline 表示子项参与所在行的基线对齐

例子

3.2.4 align-content

align-content 用于分配交叉轴(默认为垂直方向)的剩余空间，支持

- flex-start
- flex-end
- center
- space-between
- space-around
- space-evenly

  这些效果与justify-content类似，不再赘述，需要注意的是

- stretch

  在flex布局下它的默认值可以看作是`stretch`而不是`flex-start`，strech的效果是拉伸每行，以此分配剩余空间；当只有一行的时候效果就是这一行高度填满容器，当有多行的时候，会将剩余空间等比例分配给每一行；例子

  注意：align-content对`flex-wrap: nowrap`的弹性盒子模型无效

  align-content与align-items不同点可以理解为，align-content影响的是整个内容区，它的对齐参考对象是flex容器，而align-items影响的是每个子项，它的对齐参考对象是每个子项的所在行，我们可以通过调试具体例子来理解，考虑以下问题：

  为什么换行后壹和贰中间会有间隙？

https://codepen.io/dqqbl/pen/zYpMwqQ

  由于在弹性布局中没有列的概念，因此justify-items在弹性布局中是无效的

3.2.5 order

  `order: <integer>;`

  我们可以通过设置子项的order属性来改变某一个flex子项的排序位置，所有子项默认值为0，可以设置负值，例子

3.2.6 flex

CSS

```
flex: none | initial | auto | [ <'flex-grow'> <'flex-shrink'>? || <'flex-basis'> ]
```

  我们知道flex属性是三个属性的缩写，分别代表模型尺寸的放大，缩小和基础尺寸，它们的取值分别是:

```css
flex-grow: <number> /* 数值，可以是小数，默认值是 0 */
flex-shrink: <number> /* 数值，可以是小数，默认值是 1 */
flex-basis: content | <'width'>
```

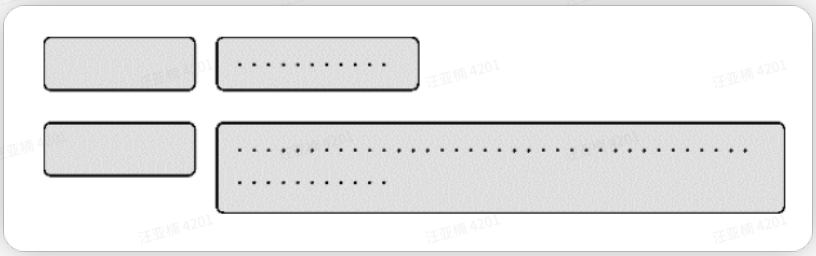
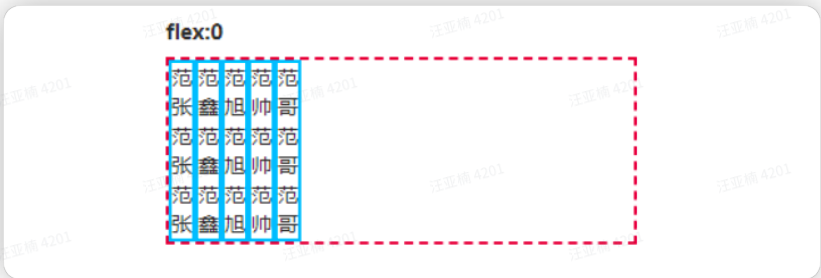
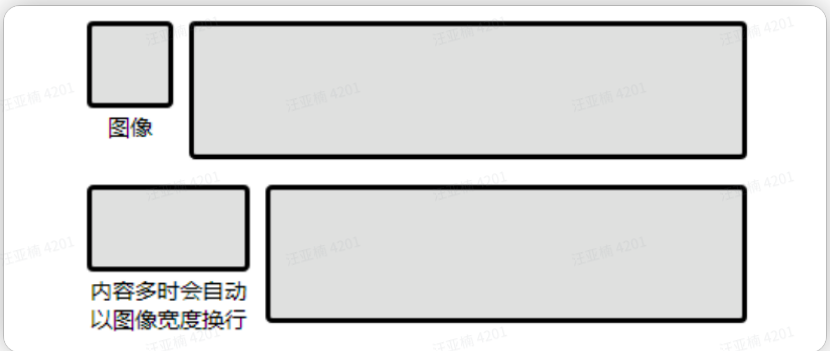
在css中，一般的多值缩写，只写其中部分值的时候，其它值通常是保持默认值，而flex不同，flex对其缩写进行了优化，当只设置了部分值的时候，其余值会被优化为符合实际情况的值

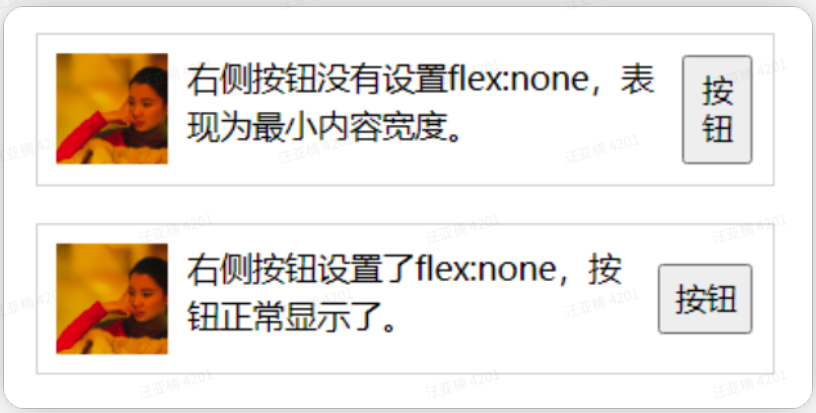
- flex-grow 指定了容器剩余空间多余时候的分配规则
  - 只有一个子项设置
    - 如果flex-grow属性值小于1，则flex子项扩展的空间就是总剩余空间和这个比例的计算值
    - 如果flex-grow属性值大于1，则flex子项独享所有剩余空间
  - 有多个子项设置
    - 如果flex-grow属性值的总和小于1，则每个flex子项扩展的空间就是总剩余空间和当前flex子项设置的flex-grow比例的计算值
    - 如果flex-grow属性值的总和大于1，则所有剩余空间被利用，分配比例就是各个flex子项的flex-grow属性值的比例
- flex-shrink 指定了容器剩余空间不足时候的分配规则
  - 如果只有1个flex子项设置
    - 如果flex-shrink属性值小于1，则收缩不完全，会有一部分内容溢出flex容器
    - 如果flex-shrink属性值大于等于1，则收缩完全，元素正好填满flex容器
  - 如果有多个子项设置
    - 如果flex-shrink属性值的总和小于1，则收缩不完全，每个元素收缩尺寸和“完全收缩的尺寸”的比例就是该元素的flex-shrink属性的值
    - 如果flex-shrink属性值的总和大于1，则收缩完全，每个元素收缩尺寸的比例和flex-shrink属性值的比例一样
- flex-basis 用于指定容器的基础尺寸，在配合width使用时情况比较复杂，这里简单总结了几条，
  - 该尺寸为保底尺寸，width和flex-shrink不能使子项尺寸小于flex-basis，但min-width可以假设flex-grow: 1，flex-shrink: 1
  - 当仅使用flex-basis时，子项尺寸取 "最小内容宽度"，"flex-basis值"，"flow-grow分配后的值"，中最大的那个
  - 当仅使用width时，子项尺寸取 "width"，"flow-grow分配后的值"中最大的那个
  - 当flex-basis和width同时使用
    - width > 最小内容宽度 > flex-basis时，子项尺寸取最小内容宽度
    - 最小内容宽度 > width > flex-basis时，子项尺寸取width
    - 最小内容宽度 > flex-basis > width时，子项尺寸取flex-basis

- flex-basis > 最小内容宽度 > width时，子项尺寸取flex-basis

可以通过调试这个例子验证

了解完这三个属性我们再来看看flex的取单值时对应这三个属性的取值

单值语法	等同于	使用场景
flex: initial	flex: 0 1 auto	初始值，auto表示尺寸由内容决定，常用于一侧固定另一侧大小自适应场景 
flex: 0	flex: 0 1 0%	不常用，0%表示初始宽度为0，尺寸表现为最小内容宽度  适用场景，以替换元素为内容主体的情况 
flex: none	flex: 0 0 auto	子项既不扩展也不缩小，且auto尺寸由内容决定，会出现内容不换行的情况

		
flex: 1	flex: 1 1 0%	二者都可以弹性缩放，两者不同点仅在于flex-basis 设置flex: 1 元素会优先牺牲自己的尺寸，适合等分布局； 设置flex: auto 元素会根据自己的内容，优先扩展自己的布局，适合基于内容动态适配的布局
flex: auto	flex: 1 1 auto	

3.3 网格布局

任意元素设置display:grid / inline-grid声明即可创建网格布局，同时会创建GFC(grid formatting context)网格格式化上下文，在网格布局容器下有以下特性：

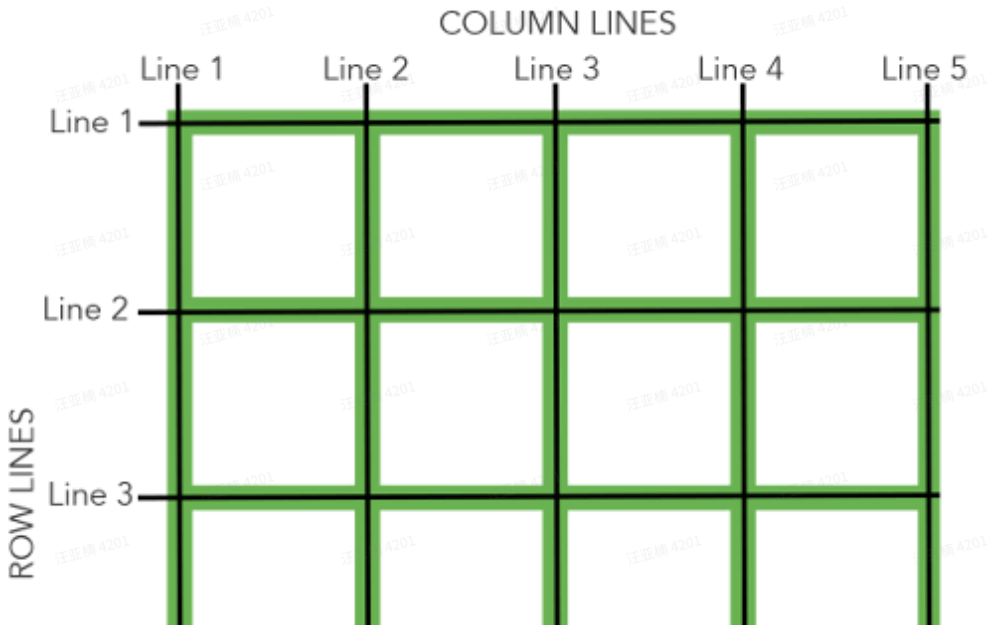
- 网格布局的子项会被块级化，如display值会改变如inline/inline-block会变为block
- 虽然子项的display值被修改为块级，但它们已经不是块级盒子而是网格级盒子，因为它们处在GFC中而不是BFC中，尺寸表现还是填满一行
- float属性失效
- gird子项支持z-index属性，当z-index不为auto时，会创建层叠上下文
- grid子项会创建BFC

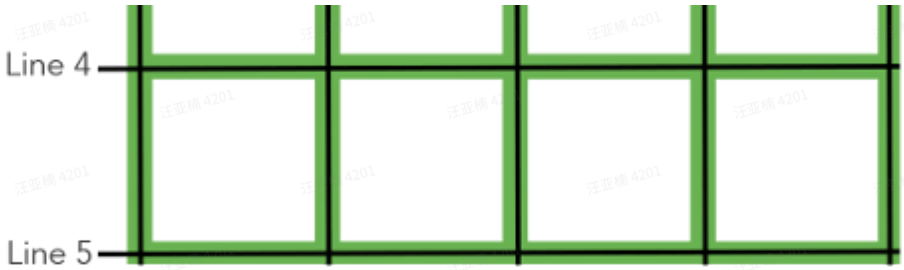
网格布局相关的属性非常多，有近30个，其中有1/3是缩写

作用在grid容器上	作用在grid子项上
grid-template-columns	grid-column-start
grid-template-rows	grid-column-end
grid-template-areas	grid-row-start
grid-template	grid-row-end
grid-column-gap	grid-column

grid-row-gap	grid-row
grid-gap	grid-area
justify-items	justify-self
align-items	align-self
place-items	place-self
justify-content	
align-content	
place-content	
grid-auto-columns	
grid-auto-rows	
grid-auto-flow	
grid	

先简单介绍下基本概念





网格布局是一种二维布局，由**水平网格线**和**垂直网格线**划分出**单元格**，水平网格线间水平区域为**行**，垂直网格线间的区域为**列**

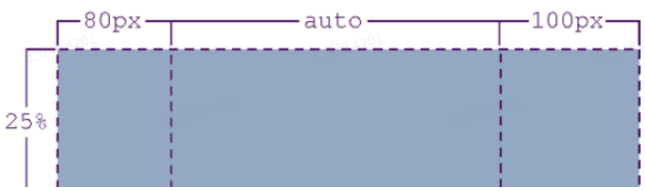
3.3.1 grid-template-columns和grid-template- rows

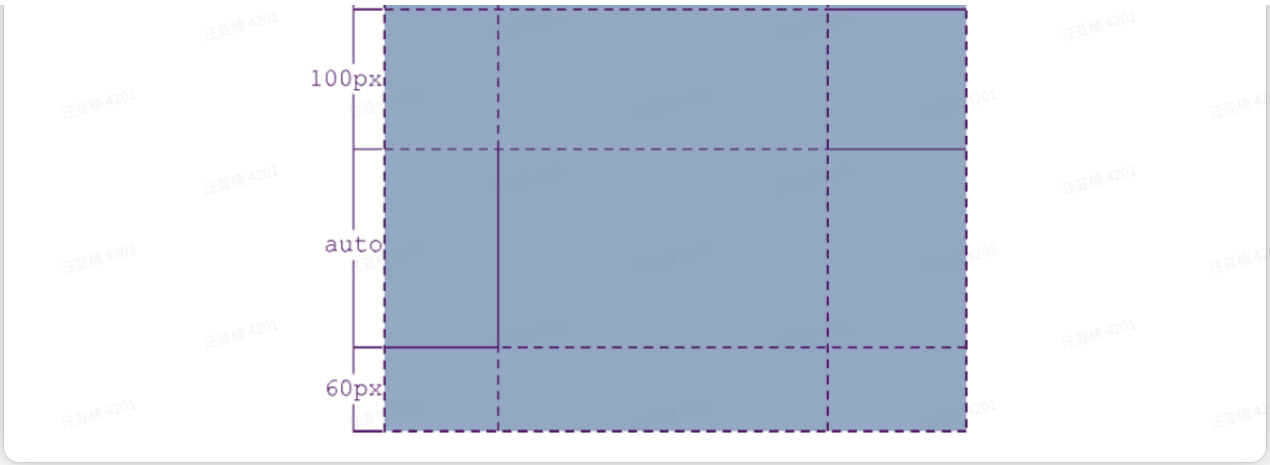
这两个属性用来指定网格的数量，尺寸，网格线名等信息，非常常用；因为默认的网格布局和块布局非常相似，仅简单的由上往下依次垂直排；这两个属性的语法是一致的，因此我们主要通过grid-template-columns学习；

CSS

```
grid-template-columns: none | <track-list> | <auto-track-list> | subgrid <line-name-list>?
/* 默认值为none */
```

```
.container {
    grid-template-columns: 80px auto 100px;
    grid-template-rows: 25% 100px auto 60px;
}
```





这是这两个属性最基本的用法，行列单元格的宽高一一对应，还可以用缩写方式:

```
CSS
grid-template: 25% 100px auto 60px / 80px auto 100px;
/* 或者 */
grid: 25% 100px auto 60px / 80px auto 100px;
```

这两个属性还可以使用中括号给网格线命名，命名后的网格线可以用于grid-column-start、grid-column-end等在子项上使用的属性上；

由于网格的中间区域的网格线是由两边格子公用的，因此，我们给网格线起名字的时候可以起两个名称(使用空格分隔)

```
CSS
grid-template-columns: [ad-left] 80px [ad-right content-left] auto [content-right tool-left] 100px [tool-right];
```

一般场景下不会使用；

grid-template-columns尺寸支持九种数据类型：

- 长度值;
- 百分比值;

- 关键字属性值，包括min-content、max-content和auto;
  - min-content、max-content我们前面学习过，分别表示列宽取所有该列单元格中最小内容宽度最大的和最大内容宽度最大的值；
  - auto默认情况下表现为拉伸，受justify-content和align-content属性影响
- <flex>数据类型，也就是以fr为单位的值;
  - fr是单词fraction的缩写，表示分数
  - 如果所有fr值之和大于1，则按fr值的比例划分可自动分配尺寸，网格元素填满容器

  - 如果所有fr值之和小于1，最终的尺寸是可自动分配尺寸和fr值的乘法计算值，即会有多余的空白尺寸是没有网格元素的

- 函数值，包括repeat()、minmax()和fit-content()
  - minmax(min, max)函数用于设置弹性尺寸，支持两个参数值，表示尺寸范围限制在min~max范围内，用法比较直观，需要注意的是fr只能用于第二个参数
  - fit-content(limit)底层计算公式为 max(minimum, min(limit, max-content))，表示尺寸由内容决定，内容越多尺寸越大，但不超过限定的尺寸
  - repeat([<positive-integer> | auto-fill | auto-fit] , <track-list>)函数的作用很简单，就是当网格尺寸可以重复的时候简化属性值的书写

CSS

```
repeat( [ <positive-integer> | auto-fill | auto-fit ] , <track-list> )

grid-template-columns: 40px auto 60px 40px auto 60px;
/* 等于 */
grid-template-columns: repeat(2, 40px auto 60px);
```

它有两个参数，第一个参数表示需要重复的次数，第二个参数表示网格轨道尺寸列表；这里主要讲下两个关键字auto-fill，auto-fit，从名字上就可以看出，auto-fill表示用尽可能多的单元格填满空间，而auto-fit表示用尽可能少的单元格填满空间，我们用例子配合调试工具可以看出不同

https://codepen.io/dqqbl/pen/QWaPLYM

还有需要注意的是repeat()函数是不能和auto一起使用的

3.3.2 grid-template-areas

grid-template-areas属性用来指定网格区域的划分，用法很直观，需要配合子项属性 `grid-area` 使用，通过 grid-area 给子项元素命名，通过grid-template-areas实现布局；`.` 符号表示空白单元格

https://codepen.io/dqqbl/pen/ZEvZEKV

需要注意的是，布局必须是矩形格式，不能是L或者凹凸形的；如果给子项命名了但没有网格线命名的话，则子项左右的网格线会被默认命名为 xx-start / xx-end，如上面的例子就是 '养猪-start' '养猪-end'

3.3.3 grid-template

grid-template属性是grid-template-rows、grid-template-columns和grid-template-areas属性的缩写，作为grid-template-rows、grid-template-columns缩写的方式上面有提到，加上grid-template-areas的时候使用比较复杂，不常用，简单了解下即可

CSS

```
.container {
  grid-template:
    [row-name1-start] '葡萄 葡萄 葡萄' 1fr
    [row-name1-end row-name2-start]
    '龙虾 养鱼 养鱼' 1fr
    [row-name2-end]
    '龙虾 养鱼 养鱼' 1fr
    [row-name3-end]
    [row-name4-start]
    '西瓜 西瓜 西瓜' 1fr [row-name4-end]
    / [col-name-start] 1fr [col-name-end] auto;
}
```

该属性可以用grid属性替换

3.3.4 grid-auto-columns和grid-auto-rows

grid-auto-columns和grid-auto-rows属性的作用是指定任何自动生成的网格(也称为隐式网格)的尺寸大小；“隐式网格”是指非正常网格，其在grid子项多于设置的单元格数量，或grid子项的位置出现在设定的网格范围之外时出现

https://codepen.io/dqqbl/pen/LYevEwm

3.3.5 grid-auto-flow

CSS

```
grid-auto-flow: [ row | column ] || dense
```

grid-auto-flow用于改变单元格流向，默认值为row;

dense 是密集的意思，如果前面空出单元格则后面的单元格会补上;

https://codepen.io/dqqbl/pen/mdpgJVg

3.3.6 grid 缩写

该缩写有四种用法：

- grid: none 表示设置所有子属性为初始值
- grid: <grid-template> 与 grid-template 的缩写一模一样

  `grid: 100px 300px / 3fr 1fr;`
- 第三第四种是类似的

CSS

```
grid: [ auto-flow && dense? ] <'grid-auto-rows'>? / <'grid-template-columns'>
grid: <'grid-template-rows'> / [ auto-flow && dense? ] <'grid-auto-columns'>?
```

它们都用在需要改变网格布局自然流向的时候，当auto-flow在斜杠左侧，它的取值为row，当auto-flow在斜杠右侧，它的取值为column，同时，根据auto-flow位置不同，斜杠两侧可能用于指定隐式网格或显式网格

```CSS
.container {
  grid: auto-flow dense 100px / 1fr 2fr;
}
/* 等同于 */
.container {
  grid-auto-flow: row dense;
  grid-auto-rows: 100px;
  grid-template-columns: 1fr 2fr;
}
```

使用grid属性时，其余所有未指明的 `grid` 次级属性都会被重置为初始值

3.3.7 区间范围设置属性

```CSS
grid-column-start: <integer> | <name> | <integer> <name> | span <number> | span <name> | auto;
grid-column-end: <integer> | <name> | <integer> <name> | span <number> | span <name> | auto;
grid-row-start: <integer> | <name> | <integer> <name> | span <number> | span <name> | auto;
grid-row-end: <integer> | <name> | <integer> <name> | span <number> | span <name> | auto;
```

这几个属性用来指定网格起止于哪条网格线；需要注意的有：

- 起止线位置可以是负数，负数则从右往左算起
- 以轨道名定位时，查找时会自动带上start / end
- span用于合并网格，`grid-column-start: span 2` 数字表示往后合并多少格，如果接的是网格线名，则表示取最接近结束线的那条命名网格线

https://codepen.io/dqqbl/pen/mdpNEmZ

3.3.8 一些通用属性在网格布局中的表现

在以前，网格间的间隙由grid-column-gap属性和grid-row-gap属性来指定，而现在间隙相关的样式，统一由 column-gap 和 row-gap 属性来设置，前面也提到了，这两个属性可以缩写为gap，只有一个值时表示行列同时设置

```CSS
gap: <row-gap> <column-gap>
```

除此之外，还有项对齐相关的属性

```CSS
justify-items: stretch | start | end | center;
align-items: normal | stretch | start | end | center | baseline;

/* 上面两个属性的缩写 */
place-items: <align-items> <justify-items>?

justify-content: normal | stretch | start | end | center | space-between | space-around | space-evenly;
align-content: normal | stretch | start | end | center | space-between | space-around | space-evenly;

/* 上面两个属性的缩写 */
place-content: <align-content> <justify-content>?

/* 设置于子项上的属性 */
justify-self: auto | normal | stretch | start | end | center | baseline;
align-self: auto | normal | stretch | start | end | center | baseline;

/* 上面两个属性的缩写 */
place-self: <align-self> <justify-self>?
```

效果与flex布局对应的值是类似的，items用于网格项的内容对齐网格线，而content用于整体的内容对齐容器;

以上就是布局css3中布局相关的内容，下面我们再来学习一下css3中用于适配设备尺寸的属性与方法

3.4 @media

@media规则，也就是我们常说的媒体查询

https://codepen.io/dqqbl/pen/rNpXmwx

@media规则主要由四部分构成，对照上面分别是修饰符only，媒体类型screen，媒体条件and，媒体特性max-width

3.4.1 媒体查询修饰符

修饰符有两个取值，only和not

- only

only修饰符是为了避免老旧浏览器只生效部分规则导致样式错乱而设计的，现在已经没有必要使用了，加不加都不影响

- not

not修饰符表示否定，它否定的并不是后面的媒体类型，而是整条查询规则，效果类似于取反

3.4.2 媒体类型

曾经支持的媒体类型数量有很多，但是我们现在只需关注三个，其余都已废弃

- print 在打印或打印预览中生效
- screen 除打印场景外的所有场景生效
- all 所有场景生效

对于打印可能的使用场景，不打印页首页尾

CSS

```
@media print {
    header, footer {
        display: none;
    }
}
```

3.4.3 媒体条件

媒体条件有三个 not，and，or 分别表示非，且，或；在最新的规范中，not只能作为修饰符使用，效果是一样的，我们将not作为查询修饰符使用即可；

CSS

```
/* 如果设备渲染更新频率慢，或者不支持鼠标行为 */
@media (update: slow) or (hover: none) {}
/* 宽度在320px～480px，同时分辨率是150dpi的设备 */
@media only screen
  and (min-width: 320px)
  and (max-width: 480px)
  and (resolution: 150dpi) {
    // ...
}
```

3.4.4 媒体特性

媒体特性指的是对媒体特性的描述信息，包括浏览器、用户设备和使用环境等特性，我们最常使用的就是min-width等尺寸相关的媒体特性，除此之外有个比较新又比较实用的特性是 prefers-color-scheme，它可以用来检测当前页面是否处于深色模式，它支持三个参数

- no-preference表示系统没有告知用户使用的颜色方案
- light表示系统倾向于使用浅色模式
- dark表示系统倾向于使用深色模式

这里介绍一个快速对网页进行深色模式改造的小技巧，利用滤镜对颜色反相，同时为了避免图片变色需要将图片再次反相，这种“偷懒”的做法比较适合不太重要的页面

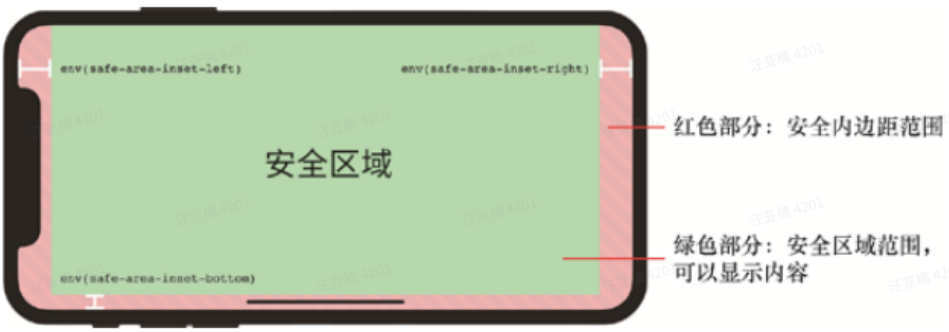
https://codepen.io/dqqbl/pen/PoEMjdP

3.5 env()

环境变量函数，与var()语法类似，不同点在于，env是将用户代理定义的环境变量值插入你的 CSS 中，而var用于自定义的变量；env()函数可以用在媒体查询语句中，甚至用在选择器中，但var()函数只能作为属性值或作为属性值的一部分；目前最常用也是支持最完善的就是用来取异形屏的安全边距，用法如下

CSS

```
/*
safe-area-inset-top
safe-area-inset-right
safe-area-inset-bottom
safe-area-inset-left
*/
padding-left: env(safe-area-inset-bottom, 50px);
```



3.6 一些适配思路

- 相对单位
  - rem 相对于根元素字体大小计算尺寸

  - vw 视区宽度百分值
  - vh 视区高度百分值
  - vmin 取vw或vh中小的那个
  - vmax 取vw或vh中大的那个
- 使用flex/grid等响应式布局，操作简单，足以应对大多数适配场景
- 动态rem，通过js计算动态改变rem
- 使用插件如 `postcss-px-to-viewport` 或 css预处理实现px转vw，这种方式可以在开发时直接写px，同时页面又会有等比例缩放的效果